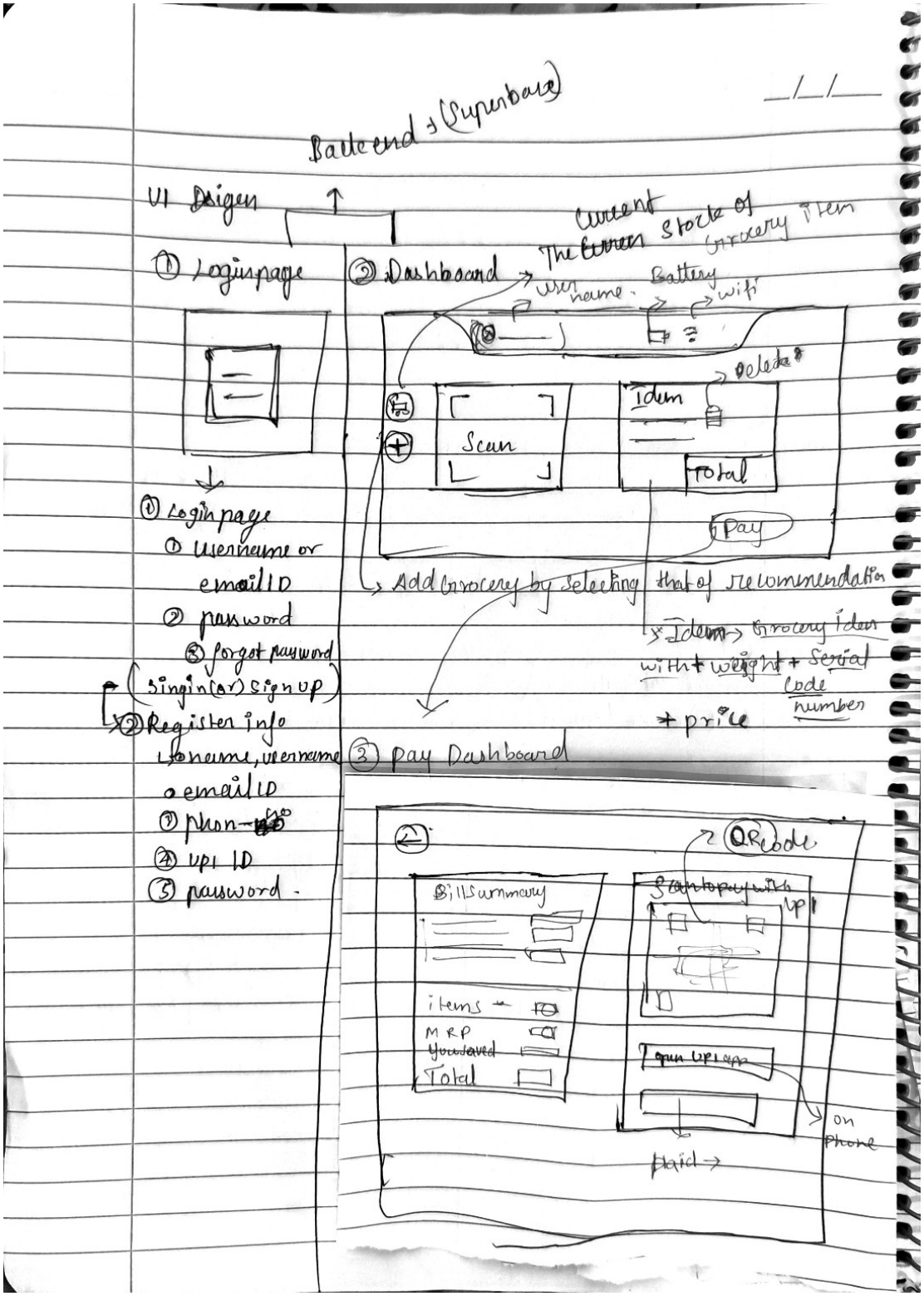
PROJECT ARCHITECTURE & USER FLOW

Explanation of the handwritten UI flow: Login/Register → Grocery Dashboard → Payment Dashboard



Project concept: a grocery checkout system where a user signs in, scans or selects grocery items, reviews item details and totals, then pays through a payment screen with QR-code support.

This document explains the components visible in the supplied sketch. Details not clearly specified in the sketch are marked as recommendations.

1. Project overview

The proposed system streamlines grocery shopping and checkout. The user authenticates, adds products through a scan or selection flow, reviews a cart with product details and prices, and proceeds to a payment dashboard. The backend stores user and grocery data and supports price lookup, cart calculations, and payment-status updates.

Main modules

Module	Purpose / contents
Backend	Server-side APIs and business logic; user authentication, grocery catalogue lookup, cart totals and payment status.
Login / Register	Sign in with username or email and password; registration captures username, email, phone number, user ID and password.
Grocery dashboard	Main shopping screen with user greeting/profile, battery indicator if this is a device-connected project, scan/add-item actions, item list and total.
Item recognition / lookup	Scan a product barcode or select a recommended product; retrieve product name, MRP, weight and barcode/product code from the catalogue.
Cart / bill summary	List selected products, quantities and prices; calculate subtotal and final total; allow removal or editing of items.
Payment dashboard	Bill summary, QR-code display or QR-based payment action, payment option/upload area if required, and final paid/pending status.

2. End-to-end user flow

Step	User action	System response
1	Open the app	Show login and registration choices.
2	Sign in or register	Validate inputs; securely verify credentials; create a session.
3	Open dashboard	Load the user profile and shopping interface.
4	Scan or add a grocery	Read barcode or selected item ID and look up product details.
5	Review cart	Show product name, MRP, weight, barcode/code, quantity and price.
6	Edit items	Allow quantity changes or remove items; recalculate totals.
7	Select Pay	Create a pending order and present the configured payment/QR workflow.
8	Confirm payment	Verify payment result, then mark the order paid or show failure/pending state.

3. Login and registration screen

Login screen

• **Username or email:** identifies the registered account.

• **Password:** authenticates the user. Store only a secure password hash on the backend.

• **Forgot password:** starts a secure reset flow using a verified email or phone.

• **Sign in / Sign up:** switch between existing-account access and registration.

Registration fields shown in the sketch

• Username and email address.

• Phone number, if required for account recovery or receipts.

• User ID: preferably generated by the backend rather than typed by the user.

• Password and password confirmation; validate strength and avoid storing plaintext.

4. Grocery dashboard

This is the main working screen after login. The sketch shows a user greeting/profile area, a scan region, add-item controls, a list of grocery entries, a delete action, a total and a Pay button.

Recommended item fields

Field	Why it is needed
Product name	Makes the recognised or selected grocery easy to identify.
Barcode / product code	Connects the physical package to a product record.
MRP	Displays the maximum retail price stored in the catalogue.
Weight / net quantity	Shows pack size, e.g. 500 g or 1 kg.
Quantity	Number of units the customer wants to buy.
Unit price and line total	Shows price per unit and quantity × unit price.
Remove / edit	Lets the user correct an incorrect scan or quantity.

Important distinction: a barcode usually identifies the product, but it does not reliably contain the current MRP or weight itself. Use the barcode to look up those values in your product database, and verify catalogue accuracy.

5. Payment dashboard

The payment dashboard is the final checkout step. The sketch includes a back control, bill summary, QR-code area, an upload/selection area and a payment status. Confirm the intended payment method before implementing the upload control; uploading a screenshot alone is not proof of payment.

Bill summary contents

• Item count and itemised product list.

• MRP or applicable unit price and quantity for each item.

• Subtotal, discounts (if any), taxes (if applicable), and final payable total.

• Order/reference number and a clear paid, pending or failed state.

QR/payment flow

• Generate or request a QR code through the selected payment provider or payment application.

• Show the exact amount and order reference to the user.

• Verify payment status through the provider's trusted response/API or server-side confirmation.

• Only mark an order as paid after verified confirmation; provide retry/cancel behavior for pending or failed payments.

6. Suggested backend and database

Table / service	Suggested data
users	user_id, username, email, phone, password_hash, created_at
products	product_id, barcode, product_name, MRP, net_weight, category, image_url
orders	order_id, user_id, status, subtotal, total, created_at
order_items	order_item_id, order_id, product_id, quantity, unit_price, line_total
payments	payment_id, order_id, provider_reference, amount, status, paid_at
API / services	Authentication, product lookup, cart validation, price calculation, order creation and payment verification.

7. Recommended implementation checklist

UI: Build Login/Register, Grocery Dashboard, Cart/Bill Summary and Payment Dashboard.

Authentication: Validate fields, hash passwords, secure sessions/tokens and implement password recovery.

Barcode scan: Use a camera or scanner to read a barcode; look up the matching product in the catalogue.

Product data: Maintain verified barcode, product name, MRP and weight values in the database.

Cart calculations: Recalculate totals on the server and validate prices/quantities before checkout.

Payment security: Use a trusted provider integration and server-side payment verification.

Error states: Handle unknown barcode, duplicate scan, offline network, invalid login, and failed/pending payment.

Testing: Test mobile responsiveness, invalid inputs, quantity edits, price changes and payment edge cases.

8. Architecture summary

User → Login/Register UI → Backend authentication → Grocery dashboard → Barcode/product lookup → Cart and bill calculation → Payment dashboard/QR → Server-side payment verification → Order confirmation.

The sketch establishes the main screens and intended journey. The exact technology stack, barcode hardware, recommendation logic, payment provider and meaning of the battery indicator are not fully specified, so those should be confirmed during implementation.